

# Agenda

- Python Syntax
- Variables & Data Types
- Operators
- Decisions
- Loops
- Lists
- Iterations

# Python Syntax

# Structure of the Code

- Basically, Python code executes from top to bottom
- Every programming language organizes code in code blocks (statements that belong together)
- Indentation in Python is very important (it's mandatory not optional)
  - Every code block (subcode) must be indented
  - Unlike other languages, code blocks do not require brackets {}
  - The number of spaces is up to you, the most common use is four, but it has to be at least one.



# Structure of the Code

- Basically, Python code executes from top to bottom
- Every programming language organizes code in code blocks (statements that belong together)
- Indentation in Python is very important (it's mandatory not optional)
  - Every code block (subcode) must be indented
  - Unlike other languages, code blocks do not require brackets {}
  - The number of spaces is up to you, the most common use is four, but it has to be at least one.

```
In [4]: print("Zeile")
        if 1 > 2:
            print("Five is greater than two!")
            print("This is a second statement.")
        else:
            print("The world is a disk!")
```

Zeile

The world is a disk!



# Structure of the Code

- Basically, Python code executes from top to bottom
- Every programming language organizes code in code blocks (statements that belong together)
- Indentation in Python is very important (it's mandatory not optional)
  - Every code block (subcode) must be indented
  - Unlike other languages, code blocks do not require brackets {}
  - The number of spaces is up to you, the most common use is four, but it has to be at least one.

```
In [4]: print("Zeile")
        if 1 > 2:
            print("Five is greater than two!")
            print("This is a second statement.")
        else:
            print("The world is a disk!")
```

```
Zeile
The world is a disk!
```

```
In [4]: if 5 > 2:
        print("Five is greater than two!")
```

Cell In[4], line 2

```
print("Five is greater than two!")
```

^

IndentationError: expected an indented block

# Important Keywords (must not be used as names)

- As with every programming language, there are some keywords that belong to the language and must not be used for names
- Some popular ones are: True, False, None, def, class, break, return

The code below prints the full list

# Important Keywords (must not be used as names)

- As with every programming language, there are some keywords that belong to the language and must not be used for names
- Some popular ones are: True, False, None, def, class, break, return

The code below prints the full list

```
In [5]: import keyword  
print(keyword.kwlist)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

# Comments

- Python has commenting capabilities for the purpose of in-code documentation.
- Comments start with a `#`, and Python will ignore them:

# Comments

- Python has commenting capabilities for the purpose of in-code documentation.
- Comments start with a #, and Python will ignore them:

```
In [11]: # This is a comment  
print("Hello, World!")
```

# Comments

- Python has commenting capabilities for the purpose of in-code documentation.
- Comments start with a #, and Python will ignore them:

```
In [11]: # This is a comment  
print("Hello, World!")
```

- Comments can be placed at the end of a line, and Python will ignore the rest of the line

# Comments

- Python has commenting capabilities for the purpose of in-code documentation.
- Comments start with a #, and Python will ignore them:

```
In [11]: # This is a comment  
print("Hello, World!")
```

- Comments can be placed at the end of a line, and Python will ignore the rest of the line

```
In [ ]: print("Hello, World!") #This is a comment
```

# Variables and Data Types

# Introducing a Variable in Python

- Variables are an essential concept to store data.
- Python has no command for declaring a variable.
- A variable is automatically created when you first assign a value to it.
- In Python, variables are not declared with any particular type.
- To output the value of a variable, we use the `print()` command.

Let's see some examples

# Introducing a Variable in Python

- Variables are an essential concept to store data.
- Python has no command for declaring a variable.
- A variable is automatically created when you first assign a value to it.
- In Python, variables are not declared with any particular type.
- To output the value of a variable, we use the `print()` command.

Let's see some examples

```
In [7]: x = 10  
        print(x)  
        x = "This is a string"  
        print(x)
```

10

This is a string

# Naming Conventions

- It is very important to choose meaningful names for variables
  - This will make your code better readable to others as well
  - You need fewer comments to explain your code
- These are the most important rules and conventions:
  - PEP 8 Style Guide for Python Code <https://peps.python.org/pep-0008/#naming-conventions>
  - A good summary can be found here:  
<https://medium.com/@rowainaabdelnasser/python-naming-conventions-10-essential-guidelines-for-clean-and-readable-code-fe80d2057bc9>

# Most Important Naming Conventions

- Names **never start with a number**
- We **don't use special characters** in names (ASCII compatibility)
- **Variable and Function Names:** Use `snake_case` (all lowercase letters with words separated by underscores).
- **Constants:** Use `ALL_CAPS` with words separated by underscores. Constants are usually defined at the top of a module.
- **Boolean Variables:** Should be named to sound like a yes/no question or statement, often starting with `is`, `has`, `can`, or `should`. E.g., `is_approved`
- **Class Names:** Use `PascalCase` (each word starts with an uppercase letter, no underscores).

# Data Types I

- Variables store values, and these values are of a particular type
- Different types exhibit different behavior
  - Adding two integers will result in the arithmetic sum
  - Adding two strings will result in a string containing both substrings
  - different data types have different functions available

# Data Types I

- Variables store values, and these values are of a particular type
- Different types exhibit different behavior
  - Adding two integers will result in the arithmetic sum
  - Adding two strings will result in a string containing both substrings
  - different data types have different functions available

In [8]:

```
x = 4
y = 5
z = x + y
print(z)

x = "Text"
y = " more text"
z = x + y
print(z)
```

9

Text more text

# Data Types II

These are the built-in data types of Python

| Type Category  | Data Types                         |
|----------------|------------------------------------|
| Text Type      | 'str'                              |
| Numeric Types  | 'int', 'float', 'complex'          |
| Sequence Types | 'list', 'tuple', 'range'           |
| Mapping Type   | 'dict'                             |
| Set Types      | 'set', 'frozenset'                 |
| Boolean Type   | 'bool'                             |
| Binary Types   | 'bytes', 'bytearray', 'memoryview' |
| None Type      | 'NoneType'                         |

Some of these we will use soon, while others you will learn throughout the semester.

# Boolean Data Type

- Boolean: `bool` has only two possible values: `True` or `False`
- In programming you often need to know if an expression is `True` or `False`.
- E.g. comparing values always returns boolean result
- Technically `False` corresponds to the value of 0, any other number is considered `True` (will be important for casting to bool)

# Boolean Data Type

- Boolean: `bool` has only two possible values: `True` or `False`
- In programming you often need to know if an expression is `True` or `False`.
- E.g. comparing values always returns boolean result
- Technically `False` corresponds to the value of 0, any other number is considered `True` (will be important for casting to bool)

```
In [10]: x = (10 < 9)
         print(x)
```

False

# Boolean Data Type

- Boolean: `bool` has only two possible values: `True` or `False`
- In programming you often need to know if an expression is `True` or `False`.
- E.g. comparing values always returns boolean result
- Technically `False` corresponds to the value of 0, any other number is considered `True` (will be important for casting to bool)

```
In [10]: x = (10 < 9)
         print(x)
```

`False`

- Boolean variables / expressions are important for decisions

# Boolean Data Type

- Boolean: `bool` has only two possible values: `True` or `False`
- In programming you often need to know if an expression is `True` or `False`.
- E.g. comparing values always returns boolean result
- Technically `False` corresponds to the value of 0, any other number is considered `True` (will be important for casting to bool)

```
In [10]: x = (10 < 9)
         print(x)
```

False

- Boolean variables / expressions are important for decisions

```
In [2]: is_true = 10 > 9
         if (is_true):
             print("10 is greater than 9")
```

10 is greater than 9

# Number Data Type

Python knows 3 numeric data types: | Type | Explanation | Example | | ----- | -----  
-----  
| ----- || `int` | A whole number, positive or negative, without decimals, of unlimited length. | 3, 88, -12837654 || `float` | Float, or "floating point number" is a number, positive or negative, containing one or more decimals. | 7.5, 1212.875620, 35e3 || `complex` | Complex numbers are written with a "j" as the imaginary part: | 3+5j |

# Number Data Type

Python knows 3 numeric data types: | Type | Explanation | Example | | ----- | -----  
-----  
| ----- || `int` | A whole number, positive or negative, without decimals, of unlimited length. | 3, 88, -12837654 || `float` | Float, or "floating point number" is a number, positive or negative, containing one or more decimals. | 7.5, 1212.875620, 35e3 || `complex` | Complex numbers are written with a "j" as the imaginary part: | 3+5j |

```
In [4]: x = 10 # int
        y = 10.0 # float needs a .0 to be a float
        z = 2 + 3j # complex
```

# Number Data Type

Python knows 3 numeric data types:

| Type                 | Explanation                                                                                           | Example                |
|----------------------|-------------------------------------------------------------------------------------------------------|------------------------|
| <code>int</code>     | A whole number, positive or negative, without decimals, of unlimited length.                          | 3, 88, -12837654       |
| <code>float</code>   | Float, or "floating point number" is a number, positive or negative, containing one or more decimals. | 7.5, 1212.875620, 35e3 |
| <code>complex</code> | Complex numbers are written with a "j" as the imaginary part:                                         | 3+5j                   |

```
In [4]: x = 10 # int
        y = 10.0 # float needs a .0 to be a float
        z = 2 + 3j # complex
```

Float can also be scientific numbers with an "e" to indicate the power of 10

# Number Data Type

Python knows 3 numeric data types:

| Type                 | Explanation                                                                                           | Example                |
|----------------------|-------------------------------------------------------------------------------------------------------|------------------------|
| <code>int</code>     | A whole number, positive or negative, without decimals, of unlimited length.                          | 3, 88, -12837654       |
| <code>float</code>   | Float, or "floating point number" is a number, positive or negative, containing one or more decimals. | 7.5, 1212.875620, 35e3 |
| <code>complex</code> | Complex numbers are written with a "j" as the imaginary part:                                         | 3+5j                   |

```
In [4]: x = 10 # int
        y = 10.0 # float needs a .0 to be a float
        z = 2 + 3j # complex
```

Float can also be scientific numbers with an "e" to indicate the power of 10

```
In [38]: x = 35e3
        print(x)
```

35000.0

# Text Data Types I

- Besides numbers, we often want to process text in our programs
- Python's built-in data type to do so is: `str`
- Strings in python are surrounded by either single quotation marks, or double quotation marks. `'hello'` can be used as well as `"hello"`.

# Text Data Types I

- Besides numbers, we often want to process text in our programmes
- Python's built-in data type to do so is: `str`
- Strings in python are surrounded by either single quotation marks, or double quotation marks. `'hello'` can be used as well as `"hello"`.

```
In [15]: my_name = "John \"guitar\" \\\n" Mayer"
         print(my_name)
         my_name = 'John'
```

John "guitar" \n Mayer

- Special characters must be "escaped" with a `\` (`\n` = return, `\t` = tabulator)
- `"` is allowed inside single quotes, `'` is allowed inside double quotes

```
In [11]: my_text = "this is 'ok' a double-quote needs an escape: \" to work"
         my_text = 'here we can use "" but \' must be escaped'
         print("this is a \nreturn")
```

this is a  
return

# Text Data Types II

- Strings can be concatenated

# Text Data Types II

- Strings can be concatenated

```
In [20]: first_name = "John"
last_name = "Doe"
full_name = first_name + "\n " + last_name
print(full_name)
```

```
John
Doe
```

# Text Data Types II

- Strings can be concatenated

```
In [20]: first_name = "John"
last_name = "Doe"
full_name = first_name + "\n " + last_name
print(full_name)
```

John  
Doe

- Strings have various functions, that can be accessed

# Text Data Types II

- Strings can be concatenated

```
In [20]: first_name = "John"
last_name = "Doe"
full_name = first_name + "\n " + last_name
print(full_name)
```

```
John
Doe
```

- Strings have various functions, that can be accessed

```
In [18]: print(full_name.upper())
print(full_name.count("oe"))
```

```
JOHN DOE
1
```

# Type Casting

- Sometimes you want to explicitly change the data type of a variable
- This process is called casting or type casting
- In Python we do have functions to do this
- Casting functions are available for all built-in datatypes, where the function name equals the type name

Here are the most important ones:

```
In [24]: a = str(3)      # Change an int value to a string
          print(type(a))
          b = int(3.7)   # Change a float value to an int
          print(b)
          c = float(3)   # Change an int value to a float
          print(c)
          c = int("3")   # Change a string value to a int
          print(c)
```

```
<class 'str'>
3
3.0
3
```

# For the Geeks: Background Knowledge I

- In Python, everything is an Object (an instance of a complex data type with functions and behavior)
- More precise, a variable points to an instance of an object
- The `is` operator checks whether two variables point to the same object in memory, not just if they have the same value.



# For the Geeks: Background Knowledge I

- In Python, everything is an Object (an instance of a complex data type with functions and behavior)
- More precise, a variable points to an instance of an object
- The `is` operator checks whether two variables point to the same object in memory, not just if they have the same value.

```
In [26]: x = 10.0  
         id(x)
```

```
Out[26]: 4562233424
```



# For the Geeks: Background Knowledge I

- In Python, everything is an Object (an instance of a complex data type with functions and behavior)
- More precise, a variable points to an instance of an object
- The `is` operator checks whether two variables point to the same object in memory, not just if they have the same value.

```
In [26]: x = 10.0  
         id(x)
```

```
Out[26]: 4562233424
```

```
In [27]: y = x  
         id(y)
```

```
Out[27]: 4562233424
```

```
In [28]: x is y
```

```
Out[28]: True
```

```
In [29]: x is 10.0
```

```
<>:1: SyntaxWarning: "is" with a literal. Did you mean "=="?  
<>:1: SyntaxWarning: "is" with a literal. Did you mean "=="?  
/var/folders/9d/kn6wbzj4d59jnkrn0n0qmbc0000gn/T/ipykernel_7412  
4/938034634.py:1: SyntaxWarning: "is" with a literal. Did you me  
an "=="?  
    x is 10.0
```

Out[29]: False

# For the Geeks: Background Knowledge II

- However, a new assignment creates a new object.



# For the Geeks: Background Knowledge II

- However, a new assignment creates a new object.

In [35]:

```
y = 20.0
print(id(y))
x = y
print(id(x))
x = 12
print(id(x))
```

```
4562233424
4562233424
4518236264
```

In [ ]:



# For the Geeks: Background Knowledge II

- However, a new assignment creates a new object.

In [35]:

```
y = 20.0
print(id(y))
x = y
print(id(x))
x = 12
print(id(x))
```

```
4562233424
4562233424
4518236264
```

In [ ]:

- Thus, variables can be reassigned to a different type at runtime (variable remains a pointer, but to a different address/object).



# For the Geeks: Background Knowledge II

- However, a new assignment creates a new object.

In [35]:

```
y = 20.0
print(id(y))
x = y
print(id(x))
x = 12
print(id(x))
```

```
4562233424
4562233424
4518236264
```

In [ ]:

- Thus, variables can be reassigned to a different type at runtime (variable remains a pointer, but to a different address/object).

In [16]:

```
x = 10
print(type(x))
x = "This is a string"
print(type(x))
```

```
<class 'int'>  
<class 'str'>
```

# Input / Output

```
print()
```

# Operators

# Types of Operators

- To work (compare, add, etc.) with variables, we need operators
- Python categorizes operators into the following groups:
  - Arithmetic operators
  - Assignment operators
  - Comparison operators
  - Logical operators
  - *Identity operators*
  - *Membership operators*
  - *Bitwise operators*

Reference: [https://www.w3schools.com/python/python\\_operators.asp](https://www.w3schools.com/python/python_operators.asp)

# Arithmetic Operators

We need arithmetic operators to calculate.

| Operator | Name           | Example             |
|----------|----------------|---------------------|
| +        | Addition       | <code>x + y</code>  |
| -        | Subtraction    | <code>x - y</code>  |
| *        | Multiplication | <code>x * y</code>  |
| /        | Division       | <code>x / y</code>  |
| %        | Modulus        | <code>x % y</code>  |
| **       | Exponentiation | <code>x ** y</code> |
| //       | Floor division | <code>x // y</code> |

# Arithmetic Operators

We need arithmetic operators to calculate.

| Operator | Name           | Example             |
|----------|----------------|---------------------|
| +        | Addition       | <code>x + y</code>  |
| -        | Subtraction    | <code>x - y</code>  |
| *        | Multiplication | <code>x * y</code>  |
| /        | Division       | <code>x / y</code>  |
| %        | Modulus        | <code>x % y</code>  |
| **       | Exponentiation | <code>x ** y</code> |
| //       | Floor division | <code>x // y</code> |

```
In [4]: 5**2
```

```
Out[4]: 25
```

# Assignment Operators

We need assignment operators to assign values to variables.

| Operator | Example | Same As   |
|----------|---------|-----------|
| =        | x = 5   | x = 5     |
| +=       | x += 3  | x = x + 3 |
| -=       | x -= 3  | x = x - 3 |
| *=       | x *= 3  | x = x * 3 |
| /=       | x /= 3  | x = x / 3 |

# Assignment Operators

We need assignment operators to assign values to variables.

| Operator | Example | Same As   |
|----------|---------|-----------|
| =        | x = 5   | x = 5     |
| +=       | x += 3  | x = x + 3 |
| -=       | x -= 3  | x = x - 3 |
| *=       | x *= 3  | x = x * 3 |
| /=       | x /= 3  | x = x / 3 |

```
In [5]: x = 5  
        x *= 2  
        print(x)
```

10

# Comparison Operators

Comparison operators are needed to make decisions. The result of a comparison operator is always `true` or `false`.

| Operator           | Name                     | Example                |
|--------------------|--------------------------|------------------------|
| <code>==</code>    | Equal                    | <code>x == y</code>    |
| <code>!=</code>    | Not equal                | <code>x != y</code>    |
| <code>&gt;</code>  | Greater than             | <code>x &gt; y</code>  |
| <code>&lt;</code>  | Less than                | <code>x &lt; y</code>  |
| <code>&gt;=</code> | Greater than or equal to | <code>x &gt;= y</code> |
| <code>&lt;=</code> | Less than or equal to    | <code>x &lt;= y</code> |

# Comparison Operators

Comparison operators are needed to make decisions. The result of a comparison operator is always `true` or `false`.

| Operator           | Name                     | Example                |
|--------------------|--------------------------|------------------------|
| <code>==</code>    | Equal                    | <code>x == y</code>    |
| <code>!=</code>    | Not equal                | <code>x != y</code>    |
| <code>&gt;</code>  | Greater than             | <code>x &gt; y</code>  |
| <code>&lt;</code>  | Less than                | <code>x &lt; y</code>  |
| <code>&gt;=</code> | Greater than or equal to | <code>x &gt;= y</code> |
| <code>&lt;=</code> | Less than or equal to    | <code>x &lt;= y</code> |

```
In [3]: 4 == 2
```

```
Out[3]: False
```

# Comparison Operators

Comparison operators are needed to make decisions. The result of a comparison operator is always `true` or `false`.

| Operator           | Name                     | Example                |
|--------------------|--------------------------|------------------------|
| <code>==</code>    | Equal                    | <code>x == y</code>    |
| <code>!=</code>    | Not equal                | <code>x != y</code>    |
| <code>&gt;</code>  | Greater than             | <code>x &gt; y</code>  |
| <code>&lt;</code>  | Less than                | <code>x &lt; y</code>  |
| <code>&gt;=</code> | Greater than or equal to | <code>x &gt;= y</code> |
| <code>&lt;=</code> | Less than or equal to    | <code>x &lt;= y</code> |

```
In [3]: 4 == 2
```

```
Out[3]: False
```

```
In [7]: 4 > 2
```

```
Out[7]: True
```

# Logical Operators

Logical operators are important if a decision is more complex and needs to consider several comparisons. The logical operators can only compare values of `True` ( $>0$ ) or `False` ( $=0$ ) and result in `True` or `False`.

Here's the table formatted in Markdown:

| Operator         | Description                                   | Example                                   |
|------------------|-----------------------------------------------|-------------------------------------------|
| <code>and</code> | Returns True if both statements are true      | <code>x &lt; 5 and x &lt; 10</code>       |
| <code>or</code>  | Returns True if one of the statements is true | <code>x &lt; 5 or x &lt; 4</code>         |
| <code>not</code> | Reverses the result; returns False if true    | <code>not (x &lt; 5 and x &lt; 10)</code> |

# Logical Operators

Logical operators are important if a decision is more complex and needs to consider several comparisons. The logical operators can only compare values of `True` ( $>0$ ) or `False` ( $=0$ ) and result in `True` or `False`.

Here's the table formatted in Markdown:

| Operator         | Description                                   | Example                                   |
|------------------|-----------------------------------------------|-------------------------------------------|
| <code>and</code> | Returns True if both statements are true      | <code>x &lt; 5 and x &lt; 10</code>       |
| <code>or</code>  | Returns True if one of the statements is true | <code>x &lt; 5 or x &lt; 4</code>         |
| <code>not</code> | Reverses the result; returns False if true    | <code>not (x &lt; 5 and x &lt; 10)</code> |

```
In [36]: x = 7
         not (x > 5 and x < 10)
```

```
Out[36]: False
```

# Control Structures

# Decisions

- One of the most common things you want to do when writing a computer program is to act upon decisions/conditions
- This is done using the 'if' statement, followed by a boolean expression, followed by a ':'
- If the statement is 'True' the code block below will be executed (indentation)

# Decisions

- One of the most common things you want to do when writing a computer program is to act upon decisions/conditions
- This is done using the 'if' statement, followed by a boolean expression, followed by a ':'
- If the statement is 'True' the code block below will be executed (indentation)

In [37]:

```
x = 4
if (x > 5):
    print("x is greater than 5")
```

# Decisions

- One of the most common things you want to do when writing a computer program is to act upon decisions/conditions
- This is done using the 'if' statement, followed by a boolean expression, followed by a ':'
- If the statement is 'True' the code block below will be executed (indentation)

In [37]:

```
x = 4
if (x > 5):
    print("x is greater than 5")
```

- Optionally, we can add zero or more `elif` parts, and the `else`

# Decisions

- One of the most common things you want to do when writing a computer program is to act upon decisions/conditions
- This is done using the 'if' statement, followed by a boolean expression, followed by a ':'
- If the statement is 'True' the code block below will be executed (indentation)

In [37]:

```
x = 4
if (x > 5):
    print("x is greater than 5")
```

- Optionally, we can add zero or more `elif` parts, and the `else`

In [3]:

```
x = 3
if x > 5:
    print("x is greater than 5")
elif x < 2:
    print("x is less than 2")
else:
    print("x between 2 and 5")
```

x between 2 and 5

# Loops

- Another very common concept in programming is loops.
- We want to repeat an action until a certain condition becomes true.
- In Python, we use the `while` statement for loops:

# Loops

- Another very common concept in programming is loops.
- We want to repeat an action until a certain condition becomes true.
- In Python, we use the `while` statement for loops:

In [19]:

```
a = 0
while a < 10:
    print(a)
    a += 2
```

```
0
2
4
6
8
```

# Lists

- We have already learned about data types.
- Python has several types to manage a lists of data (this is one of Python's strengths)
- Let's introduce the most common sequence type: `list`
- Lists are used to store multiple items in a single variable.
- A list is ordered, this means that each value has an exact position in the list.
- Lists are created using square brackets.

# Lists

- We have already learned about data types.
- Python has several types to manage a lists of data (this is one of Python's strengths)
- Let's introduce the most common sequence type: `list`
- Lists are used to store multiple items in a single variable.
- A list is ordered, this means that each value has an exact position in the list.
- Lists are created using square brackets.

```
In [20]: fruits = ["apple", "orange", "grape"]
```

# Lists

- We have already learned about data types.
- Python has several types to manage a lists of data (this is one of Python's strengths)
- Let's introduce the most common sequence type: `list`
- Lists are used to store multiple items in a single variable.
- A list is ordered, this means that each value has an exact position in the list.
- Lists are created using square brackets.

```
In [20]: fruits = ["apple", "orange", "grape"]
```

- An element in a list can be accessed by the variable, followed by the index (position) in `[]`
- The first element has the index 0

# Lists

- We have already learned about data types.
- Python has several types to manage a lists of data (this is one of Python's strengths)
- Let's introduce the most common sequence type: `list`
- Lists are used to store multiple items in a single variable.
- A list is ordered, this means that each value has an exact position in the list.
- Lists are created using square brackets.

```
In [20]: fruits = ["apple", "orange", "grape"]
```

- An element in a list can be accessed by the variable, followed by the index (position) in `[]`
- The first element has the index 0

```
In [23]: apple = fruits[0]  
print(fruits[1])
```

orange

# Working with Lists

- A list object has various functions to query and modify the list.

Here are some examples of common list actions:

# Working with Lists

- A list object has various functions to query and modify the list.

Here are some examples of common list actions:

```
In [39]: fruits = ["apple", "orange", "grape", "orange"]
fruits.append("banana") # add a new element to the end
print(fruits)
fruits.insert(1, "cherry") # add a new element at index 1
print(fruits)
fruits.pop(2) # remove element at position 2
print(fruits)
fruits.sort() # sort the list
same_fruits = fruits.copy() # returns a copy of the list
print(fruits)
```

```
['apple', 'orange', 'grape', 'orange', 'banana']
['apple', 'cherry', 'orange', 'grape', 'orange', 'banana']
['apple', 'cherry', 'grape', 'orange', 'banana']
['apple', 'banana', 'cherry', 'grape', 'orange']
```

# Working with Lists

- A list object has various functions to query and modify the list.

Here are some examples of common list actions:

```
In [39]: fruits = ["apple", "orange", "grape", "orange"]
fruits.append("banana") # add a new element to the end
print(fruits)
fruits.insert(1, "cherry") # add a new element at index 1
print(fruits)
fruits.pop(2) # remove element at position 2
print(fruits)
fruits.sort() # sort the list
same_fruits = fruits.copy() # returns a copy of the list
print(fruits)
```

```
['apple', 'orange', 'grape', 'orange', 'banana']
['apple', 'cherry', 'orange', 'grape', 'orange', 'banana']
['apple', 'cherry', 'grape', 'orange', 'banana']
['apple', 'banana', 'cherry', 'grape', 'orange']
```

- Unlike arrays in other programming languages, Python's lists can store mixed data types

# Working with Lists

- A list object has various functions to query and modify the list.

Here are some examples of common list actions:

```
In [39]: fruits = ["apple", "orange", "grape", "orange"]
fruits.append("banana") # add a new element to the end
print(fruits)
fruits.insert(1, "cherry") # add a new element at index 1
print(fruits)
fruits.pop(2) # remove element at position 2
print(fruits)
fruits.sort() # sort the list
same_fruits = fruits.copy() # returns a copy of the list
print(fruits)
```

```
['apple', 'orange', 'grape', 'orange', 'banana']
['apple', 'cherry', 'orange', 'grape', 'orange', 'banana']
['apple', 'cherry', 'grape', 'orange', 'banana']
['apple', 'banana', 'cherry', 'grape', 'orange']
```

- Unlike arrays in other programming languages, Python's lists can store mixed data types

```
In [25]: list1 = ["abc", 34, True, 40, "male"]
```

# Iterations I

- When working with lists, we often want to iterate through all elements of a list
- In Python we use the `for ... in ...:` statement for this
  - The code block will be executed for each element in the list
  - The variable before the `in` will be assigned the current element of the list.
  - The behind the `'in'` is the list being referenced.

# Iterations I

- When working with lists, we often want to iterate through all elements of a list
- In Python we use the `for ... in ...:` statement for this
  - The code block will be executed for each element in the list
  - The variable before the `in` will be assigned the current element of the list.
  - The behind the `'in'` is the list being referenced.

```
In [12]: fruits = ["apple", "orange", "grape", "banana"]  
         for fruit in fruits:  
             print(fruit)
```

```
apple  
orange  
grape  
banana
```

# Iterations II

- Sometimes we don't have a list, but want to iterate over a range of numbers, e.g. from 0 to 10
- In Python we have the `range()` function to create a list of arithmetic progressions

# Iterations II

- Sometimes we don't have a list, but want to iterate over a range of numbers, e.g. from 0 to 10
- In Python we have the `range()` function to create a list of arithmetic progressions

```
In [32]: list = range(5) # Creates a list with 5 values, starting from 0
list = range(5, 10) # Creates a list with int from 5 to 9 (interval 5,
list = range(0, 10, 3) # Creates a list of values between 0 and 10 with
```

# Iterations II

- Sometimes we don't have a list, but want to iterate over a range of numbers, e.g. from 0 to 10
- In Python we have the `range()` function to create a list of arithmetic progressions

```
In [32]: list = range(5) # Creates a list with 5 values, starting from 0
list = range(5, 10) # Creates a list with int from 5 to 9 (interval 5,
list = range(0, 10, 3) # Creates a list of values between 0 and 10 with
```

- We can use `range()` to build 'for' loops

# Iterations II

- Sometimes we don't have a list, but want to iterate over a range of numbers, e.g. from 0 to 10
- In Python we have the `range()` function to create a list of arithmetic progressions

```
In [32]: list = range(5) # Creates a list with 5 values, starting from 0
list = range(5, 10) # Creates a list with int from 5 to 9 (interval 5,
list = range(0, 10, 3) # Creates a list of values between 0 and 10 with
```

- We can use `range()` to build 'for' loops

```
In [35]: for i in range(0, 8, 2): # in C++ this would be for (int i=0; i<10; i+=
print(i)
```

```
0
2
4
6
```

# Breaking Loops

- Sometimes we want to break out of a loop to save time
- In Python, we have two statements to do so, they work with `for` and `while` loops
- `break` jumps out of the complete loop and continues after the loop statement
- `continue` continues with the next iteration of the loop

# Breaking Loops

- Sometimes we want to break out of a loop to save time
- In Python, we have two statements to do so, they work with `for` and `while` loops
- `break` jumps out of the complete loop and continues after the loop statement
- `continue` continues with the next iteration of the loop

```
In [37]: for i in range(10):  
         if i == 1:  
             continue # go to the next loop, don't execute code below  
         if i == 4:  
             break # completely jump out of the loop  
         print(i)  
print("This is the next line of code after the loop")
```

0

2

3

This is the next line of code after the loop

